

Chess Provers in Lean

Malhar Patel Aastha Gupta Rajlakshmi Chavan Saachi Kaup

May 23, 2026

Agenda

- Current System
- Alternative System

Approach Followed

- Built a proof-aware chess validation system in Lean.
- Input FEN, parse LLM-proposed moves, verify legality in Lean.
- Success only when the validated sequence ends in checkmate.
- Design split: heuristic generation (LLM), formal acceptance (Lean rules + proofs).

System Map: Main.lean

- Entry flow in `main`:
 - 1 Parse input FEN using `fenToGameState`
 - 2 Call Python client: `script.py <fen>`
 - 3 Parse JSON response via `parsePythonOutputToMoves`
 - 4 Validate sequence with `state.isValidMoves`
 - 5 Execute `playMoves` and check `isAnyCheckmate`
- Output includes success/failure and board-state sequence for valid mates.

Trusting Initial State: Chess/FEN.lean

- Example FEN:
`r2k2nr/pp1b1Q1p/2n4b/3N4/3q4/3P4/PPP3PP/4RR1K w - - 1 0`
- `fenToGameState` has the following helpers:
`charToPiece`, `expandFenDigit`, `parseRankToVector`,
`parseBoardToVector`, `parseCastling`, `parseEnPassant`.
- Invalid FEN sets `valid := false` with a safe fallback board/state.

LLM Output to Typed Moves: Chess/MoveParser.lean

- JSON schema represented by `JsonMove` and `JsonCheckmateSolution`.
- Conversion chain:
 - string decoders: `stringToPiece`, `stringToColour`, `stringToPos`
 - structural conversion: `convertMoves : List JsonMove -> List Move`
- `parsePythonOutputToMoves` is the typed boundary from untrusted JSON to Lean moves.

Model Core (1): Data Types and Pseudo-Legality

- In `Chess/Model.lean`: core types `GameState`, `Move`, `Board`, `Pos`.
- Piece-specific move semantics encoded in:
 - `isValidPawnMove`, `isValidKnightMove`, `isValidBishopMove`, etc.
- `isPseudoLegalMove` enforces geometry/path/ownership constraints, but not king-safety after move.

Model Core (2): Check, Legality, and Play Pipeline

- `isInCheck`: determines whether king is attacked.
- `isLegalMove`: strengthens pseudo-legality with king safety.
- State-transition and sequence validators:
 - `makeValidMove`
 - `isValidMoves`
 - `playMoves`
- End condition functions:
 - `isCheckmate`
 - `isAnyCheckmate`

- Focus theorem: `legal_implies_kingSafeAfter`
- Meaning: if `isLegalMove s m = true`, then `isInCheck (s.makeMove m) s.turn = false`.
- This links boolean legality checks to a concrete king-safety guarantee.

- `no_friendly_fire`: valid target squares never contain a same-color piece.
- `bishop_stays_on_colour`: bishop moves preserve board-square parity (color class).
- Role: demonstrate local rule invariants and movement properties as machine-checked facts.

Demo

Alternative Approach from Mal-Pat/Chess-Learn

- Repo centers around `ChessLean/Basic.lean`.
- Architecture leans toward internal move generation:
 - `generateAllPseudoLegalMoves`
 - `IsPseudoLegalMove` as membership
 - `IsLegalMove := pseudo-legal && not-in-check`
- Execution style: `playMove/play` over generated legal-space transitions.

- We implemented a practical hybrid: external generation, internal formal acceptance.
- Proofs already anchor key safety and movement invariants.
- Next scope can expand from checkmate solving to broader verified chess reasoning.

Thank You